دليل التشغيل وPostman والمقابلة

هذه النسخة تشمل Laravel للتطبيق الرئيسي وGo للمستودع، وعقد gRPC مستقلًا، وقاعدتي PostgreSQL، وواجهة السلة وإدارة الكمية والإشعارات. ملفات العرض المتصلة بقاعدة جهازك تُولد عند التشغيل؛ لا نضع كلمات مرور ثابتة أو معرّفات منتجات من جهاز آخر في الحزمة.

1. تطبيق التحديث والتشغيل

1. افتح Docker Desktop وانتظر جاهزيته. يلزم Python 3.10 أو أحدث وPostman Desktop.
2. فك ZIP في مجلد مستقل. ادخل المجلد الذي يحتوي `compose.yaml` و`Start-Shop.ps1`.
3. إذا كانت لديك نسخة تعمل وتريد بياناتها، انسخ ملف `.env` القديم إلى جذر النسخة الجديدة. حافظ على Docker volumes. اسم مشروع Compose هو نفسه `shop-laravel`، ولذلك تستخدم هذه النسخة بيانات المشروع المحلية السابقة.
4. افتح PowerShell من شريط عنوان مستكشف الملفات بكتابة `powershell`، ثم نفّذ:

```
powershell -ExecutionPolicy Bypass -File .\Start-Shop.ps1
```

ينشئ الأمر الإعدادات والمفاتيح إن كانت ناقصة، ويستعيد إعدادات الحاويات الموجودة عند غياب `.env`، ثم يبني صورتي التطبيق والمستودع ويشغّل الخدمات ويتحقق من الاتصال على 50051 و50052. البناء الأول قد يستغرق عدة دقائق لتجميع امتداد PHP gRPC؛ البناء التالي يستفيد من ذاكرة Docker المؤقتة.

إذا طُلبت كلمة مرور المسؤول، أدخل الكلمة التي تستخدمها في المتصفح عند وجود قاعدة قديمة. في تثبيت جديد اختر كلمة مرور من 8 إلى 72 بايت. تظل كلمة المرور في `.env` المحلي. `ADMIN_PASSWORD` لا يعيد تعيين حساب موجود.

يفتح الأمر المتجر، ويفتح مجلد `Postman-Ready`. افتراضيًا:

العنصر	العنوان
المتجر	`http://127.0.0.1:8080`
التوثيق	`http://127.0.0.1:8080/docs`
OpenAPI	`http://127.0.0.1:8080/openapi/v1.json`
gRPC للمستودع	`127.0.0.1:50051`
المسؤول	`admin@shop.local` ما لم تغيّره في `.env`

إن تغيّرت المنافذ، اعتمد العناوين التي يطبعها أمر التشغيل. لا تغيّر ملف `.env` في التطبيق الداخلي؛ الإعداد المعتمد هو الملف في جذر الحزمة.

للتشغيل التالي دون تحديث الكود:

```
powershell -ExecutionPolicy Bypass -File .\Start-Shop.ps1 -SkipBuild
```

لتجهيز بيانات جديدة دون إعادة البناء أو التشغيل:

```
powershell -ExecutionPolicy Bypass -File .\Prepare-Postman.ps1
```

يُنشئ كل تجهيز منتجًا وطلبًا وحسابَي عرض جديدين، ويستبدل ملفات `Postman-Ready` بأحدث سيناريو. استورد المجموعة الجديدة بعد إعادة التجهيز. الملفات المولدة تحتوي بيانات الدخول المحلية؛ شارك ZIP مع الفريق، واطلب من كل جهاز توليد سيناريوهاته عليه.

2. اختبار REST الجاهز في Postman

افتح `Postman-Ready` واسحب إلى Postman الملفين:

- `shop-rest-ready.postman_collection.json`
- `shop-operations-ready.postman_collection.json`

تحتوي المجموعتان قيم العنوان والمنتج والطلب وبيانات حسابات العرض داخلهما. تعملان مع **No environment**، وتعطيان الأولوية لبياناتهما حتى لو اخترت بيئة قديمة. لا تعدّل كلمات المرور ولا تنقل بيانات سيناريو قديم من جهاز آخر.

افتح مجموعة **Shop Ready - REST …** ثم **Run**، واجعل Iterations = 1 وشغّل الطلبات الثمانية بالترتيب. النتيجة الصحيحة **13 تحققًا ناجحًا**:

الرقم	الاختبار	النتيجة المتوقعة
01	Laravel health	200
02	تفاصيل المنتج والرصيد الفعلي	200 والكمية 3
03	دخول صاحب الطلب	200 وحفظ JWT تلقائيًا
04	قراءة صاحب الطلب لطلبه	200
05	محاولة الزبون تغيير حالة الطلب	403، وهذا نجاح لاختبار الصلاحيات
06	دخول زبون آخر	200
07	محاولة الزبون الآخر قراءة الطلب	404، وهذا نجاح لاختبار الملكية
08	إشعار الطلب عند صاحبه	200 ووجود الإشعار

اترك 60 ثانية بين تشغيل المجموعات أو إعادة اختبارات الدخول؛ الحماية الافتراضية خمس محاولات في الدقيقة لكل IP. إذا ظهر 429، انتظر دقيقة وأعد المجموعة من أولها. لا تتكرر محاولات كلمة مرور خاطئة؛ خمس محاولات فاشلة تقفل الحساب خمس دقائق.

تجربة إنشاء المنتج والكمية والطلب والإلغاء

اختر **Shop Ready - Operations**. في الطلب رقم 03 افتح **Body** وتحقق من الصورة في الحقل `image`. إن طلب Postman إذن وصول للملف أو أظهر أن الملف غير موجود، اختر `Postman-Ready/demo-product.png` من جهازك. يمكن أيضًا اختيار مجلد `Postman-Ready` بوصفه working directory في إعدادات الملفات. جهّز اختيار الصورة قبل الدخول إلى المقابلة. لا تضبط Content-Type يدويًا لطلب multipart؛ يضيف Postman قيمة boundary الصحيحة.

شغّل المجموعة كاملة وبالترتيب، Iterations = 1. تحتوي **23 طلبًا و28 تحققًا**:

الأرقام	ما يحدث
01-03	دخول المسؤول، قراءة التصنيفات، وإنشاء منتج بصورة
04-06	إضافة 10 قطع، تكرار الإضافة بالمعرّف نفسه، والتحقق أن الرصيد بقي 10
07-10	التصفية وتقسيم الصفحات، دخول العميل وبياناته ورفض تعديل المخزون منه
11-15	إنشاء طلب قطعتين بحالة Pending، الرصيد يصبح 8، وعرض الطلبات ورفض تغيير الحالة من الزبون
16-19	دخول الزبون الآخر، رفض قراءة طلب الغير، ورفض طلب 99 قطعة دون تغيير الرصيد
20-23	تأكيد المسؤول ثم إلغاء الطلب، عودة الرصيد إلى 10، وظهور إشعار الإلغاء

المجموعة تنشئ منتجًا جديدًا في كل تشغيل كامل، ولذلك يمكن تكرارها من البداية. لا تعد الطلب 11 وحده مرات كثيرة ثم تتوقع الأرصدة نفسها؛ ذلك ينشئ طلبات إضافية. لإنشاء حساب جديد يدويًا استخدم `POST /api/v1/auth/register` بجسم يحوي `email` و`password`، وكلمة مرور قوية مثل نمط `DemoCustomer123!`، ثم سجّل الدخول.

3. اختبار gRPC عمليًا

1. في Postman اختر **New > gRPC**. إن كان لديك تبويب HTTP، غيّر نوع الطلب من رمز البروتوكول بجانب اسم الطلب؛ قائمة GET هي لطرق HTTP فقط.
2. ضع العنوان `127.0.0.1:50051` أو العنوان المطبوع في التجهيز. أبقِ TLS معطلًا للعرض المحلي.
3. افتح **Service definition > Import a .proto file** واختر `Postman-Ready/inventory.proto`، وهو نسخة مطابقة للعقد الأصلي في `contracts/inventory/v1/inventory.proto`.
4. اختر الطريقة من القائمة بجانب عنوان الخادم، وافتح **Message** والصق محتوى ملف JSON المقابل. افتح الملف بمحرر نصوص، وانسخ المحتوى كله بما فيه الأقواس.
5. اضغط **Invoke** واحفظ كل طلب بعد ضبط طريقته ورسالته.

استخدم `Postman-Ready/grpc-rehearsal` للتدريب، ثم `Postman-Ready/grpc` للمقابلة. مجموعتا البيانات مستقلتان. لا تستورد ملفات رسائل JSON بوصفها Collection؛ الصق محتواها داخل Message للطلب الموافق.

الترتيب	الطريقة	الملف	النتيجة
1	GetStock	GetStock.json	OK 0، والمتاح 3
2	Reserve	Reserve.json	reserved = true
3	GetStock	GetStock.json	المتاح 2
4	Reserve	Reserve.json مرة ثانية	يظل المتاح 2؛ لا يتكرر الخصم
5	Release	Release.json	OK 0، وقد يكون الجسم فارغًا
6	GetStock	GetStock.json	المتاح 3
7	Release	Release.json مرة ثانية	المتاح يبقى 3؛ لا تتكرر الإضافة
8	Reserve	Insufficient.json	shortages لطلب 99 قطعة، والمتاح يبقى 3
9	Reserve	AtomicShortage.json	نقص في المنتج الثاني، ولا تُخصم القطعة المتوفرة من الأول

الحقول في رسائل gRPC هي `product_id` و`order_id`، بينما حقول REST مثل `productId` هي camelCase. انسخ الملفات المولدة بدل كتابة UUID يدويًا. رفض الكمية حالة عمل تجارية داخل رد gRPC ناجح OK 0؛ يتحول في REST إلى 409. قد يخفي Postman الحقل `reserved` عندما تكون قيمته الافتراضية false؛ اقرأ `shortages` وأعد GetStock.

بعد إرجاع حجز، لا تستخدم معرّف طلبه نفسه لحجز جديد: الإرجاع يترك سجلًا يمنع تكرار الحجز المتأخر. لتكرار العرض استخدم Prepare-Postman لسيناريو جديد. احفظ طلبات gRPC قبل المقابلة؛ لا تعتمد على تشغيل مجموعة gRPC آليًا ضمن خطة مدفوعة.

4. اختبار Race Condition لعشرين عميلًا

نفّذ والمشروع يعمل:

```
powershell -ExecutionPolicy Bypass -File .\Test-Race.ps1
```

ينشئ الاختبار منتجًا مستقلًا بقطعة واحدة و20 حسابًا مختلفًا، ويسجل دخولهم، ثم يطلق 20 طلب شراء بالتوازي بإشارة بدء مشتركة باستخدام ThreadPoolExecutor وBarrier في Python. ليس مجرد 20 تكرارًا متسلسلًا في Collection Runner.

قد يستغرق تسجيل الحسابات نحو أربع إلى خمس دقائق بسبب حماية الدخول؛ ينتظر السكربت تلقائيًا عند 429. بعد جاهزيتها يبدأ السباق الفعلي سريعًا. لا تحتاج حزم Python إضافية. تظل حماية التطبيق مفعلة.

النتيجة المطلوبة:

```
HTTP results: {201: 1, 409: 19}
Available units after the race: 0
PASS: 20 customers, 1 successful order, 19 stock conflicts, zero available, winner Pending.
```

قد يظهر ترتيب 201 و409 بالعكس؛ المهم الأعداد. يُحفظ تقرير آمن دون كلمات مرور أو رموز دخول في `Postman-Ready/race-*.demo/race-…/results.json` وكذلك `results.json`. يتضمن التقرير المنتج والطلب الفائز ونتيجة كل محاولة. لا يشغّل الاختبار على بضاعتك الفعلية؛ ينشئ بيانات عرض منفصلة ولا يوقف الخدمات.

تجهيز الحسابات مسبقًا ثم إطلاق السباق أمام الدكتورة

قبل العرض بقليل:

```
powershell -ExecutionPolicy Bypass -File .\Test-Race.ps1 -PrepareOnly
```

عند بدء العرض، من PowerShell في جذر المشروع:

```
$race = (Get-Content .\Postman-Ready\race-latest.txt -Raw).Trim()
powershell -ExecutionPolicy Bypass -File .\Test-Race.ps1 -Prepared $race
```

رموز الدخول صالحة 15 دقيقة من تسجيل الدخول؛ استخدم التجهيز بعد اكتماله مباشرة. إذا انتهت صلاحية رمز، يعيد الاختبار تسجيل دخوله مع احترام الحد. تُستخدم كل حالة سباق مرة واحدة، وبعدها جهّز حالة جديدة.

لماذا ينجح واحد فقط؟ يحجز Go داخل transaction، ويقفل صفوف المخزون بـ SELECT … FOR UPDATE بترتيب ثابت. أول معاملة تخصم القطعة وتنفذ COMMIT؛ تقرأ المحاولة التالية الرصيد صفرًا بعد انتظار القفل، فتعيد النقص دون خصم. يوجد أيضًا CHECK (available >= 0)، لكن القفل والمعاملة هما أساس منع السباق.

هل يحجز وضع المنتج في السلة؟ لا. الحجز يحدث عند إرسال الطلب، قبل حفظه وقبل موافقة المسؤول. طلب Pending يحتفظ بالكميات حسب نص المتطلب الثاني؛ وإلغاء المسؤول يعيدها. انتظار موافقة المسؤول للحجز يتيح بيع الكمية نفسها عدة مرات.

5. إيقاف الخدمات وقواعد البيانات وإعادتها

نفّذ الأوامر من جذر المشروع. كل الأوامر التالية تحافظ على volumes والبيانات.

المطلوب	الإيقاف	الإعادة
خدمة gRPC فقط	`docker compose stop warehouse`	`docker compose -f compose.yaml -f compose.postman.yaml up -d --wait --wait-timeout 180 warehouse`
قاعدة المستودع فقط	`docker compose stop warehouse-db`	`docker compose up -d --wait --wait-timeout 180 warehouse-db`
قاعدة تطبيق Laravel فقط	`docker compose stop postgres`	`docker compose up -d --wait --wait-timeout 180 postgres`
كل المشروع	`docker compose stop`	`docker compose -f compose.yaml -f compose.postman.yaml up -d --wait --wait-timeout 180`

مثال عرض فشل المستودع:

```
docker compose stop warehouse
# في 02 الآن أرسل الطلب Shop Ready - REST: متوقع HTTP 503.
docker compose -f compose.yaml -f compose.postman.yaml up -d --wait --wait-timeout 180 warehouse
# متوقع: 02 أعد الطلب HTTP 200 والرصيد السابق.
```

إيقاف قاعدة المستودع مع إبقاء خادم gRPC يعمل:

```
docker compose stop warehouse-db
# GetStock في Postman: متوقع gRPC UNAVAILABLE، وطلب تفاصيل المنتج يعيد REST 503.
docker compose up -d --wait --wait-timeout 180 warehouse-db
# أعد GetStock وتفاصيل المنتج بعد عودة جاهزية القاعدة.
```

يُختبر انتهاء المهلة تحديدًا بتعليق الخدمة مع بقاء اتصالها مفتوحًا:

```
docker compose pause warehouse
# طلب تفاصيل المنتج عبر REST: 503 بعد مهلة نداء تقارب ثانيتين، مع هامش معالجة الطلب.
docker compose unpause warehouse
```

لا تستخدم stop ولا up لإعادة خدمة معلقة؛ استخدم unpause أولًا. اختبار `/health` أو اتصال TCP وحده لا يثبت أن قاعدة المستودع تعمل؛ اختبر GetStock لأنّه ينفذ استعلامًا فعليًا. قد تبقى حاوية warehouse ظاهرة healthy رغم توقف قاعدتها، لأن فحصها الحالي يتحقق من خادم gRPC.

قاعدة التطبيق `postgres` تحفظ المستخدمين والمنتجات والطلبات والإشعارات؛ إيقافها يعطل العمليات التي تحتاج تلك الجداول. قاعدة `warehouse-db` تحفظ المخزون والحجوزات فقط. إذا سُئلت عن إيقاف المشروع كله:

```
powershell -ExecutionPolicy Bypass -File .\Stop-Shop.ps1
```

للتشخيص دون تغيير بيانات:

```
docker compose -f compose.yaml -f compose.postman.yaml ps -a
docker compose logs --no-color --tail 60 api warehouse warehouse-db
```

أمر `docker compose down` يزيل الحاويات والشبكات، وتبقى البيانات في named volumes افتراضيًا. إضافة `-v` تحذف volumes وبيانات المشروع؛ لا تحتاجها للتشغيل ولا لعرض حالات الفشل. لا تحذف `.env` ما دمت تريد استخدام القواعد السابقة.

6. أين يوجد كود gRPC؟

كل المسارات التالية تبدأ من جذر المشروع:

الملف أو المجلد	وظيفته
`contracts/inventory/v1/inventory.proto`	العقد: InventoryService وGetStock وReserve وRelease والرسائل
`warehouse/cmd/warehouse/main.go`	خادم Go، تنفيذ العمليات، الجداول، المعاملات وأقفال الصفوف
`warehouse/gen/inventory/v1/`	كود Go المولّد من Protobuf وتسجيل الخدمة والواجهات
`laravel/app/Modules/Warehouse/Inventory.php`	واجهة مخزون تستخدمها وحدات التطبيق
`laravel/app/Modules/Warehouse/GrpcInventory.php`	عميل Laravel الفعلي؛ يبني الرسائل ويرسل RPC ويطبق timeout ويحوّل الأخطاء
`laravel/generated/Inventory/V1/`	رسائل Protobuf وعميل PHP المولّدان
`laravel/app/Modules/Orders/OrderService.php`	جمع بنود السلة، Reserve واحد، حفظ الطلب والإشعار، والإلغاء
`laravel/app/Modules/Orders/InventoryRecovery.php`	سجل الاسترداد ومحاولة Release عند الفشل أو الإلغاء
`laravel/app/Modules/Products/ProductController.php`	إنشاء المنتج، تفاصيله مع الرصيد، وإضافة الكمية للمسؤول
`contracts/inventory/v1/inventory_admin.proto`	عقد AddStock الإداري المنفصل على 50052 داخل شبكة Docker
`tests/concurrent_checkout.py`	تجهيز 20 حسابًا واختبار التزامن وتقرير النتائج
`tests/laravel_acceptance.py`	فشل حفظ الطلب بعد الحجز، التعويض، المهلة، والانقطاع والصلاحيات

افتح main.go وابحث عن:

- `func (w *warehouse) GetStock` : قراءة الكمية.
- `func (w *warehouse) Reserve` : معاملة واحدة، تحقق من جميع البنود، ثم الخصم.
- `func (w *warehouse) Release` : إعادة المخزون مرة واحدة فقط.
- `func addStock` و`AddStock` : إضافة إدارية مع operation_id يمنع التكرار.
- `FOR UPDATE` و`normalize` : قفل الصفوف وترتيب المعرّفات.

7. إجابات قصيرة للمقابلة

ما gRPC؟ استدعاء وظائف بين خدمات عبر شبكة باستخدام عقد Protobuf وHTTP/2. Laravel يستدعي وظائف Go مباشرة وفق العقد، ولا يتصل بقاعدة المستودع.

لماذا Go مع Laravel؟ يحقق اختلاف اللغة المطلوب، وله دعم رسمي لـ gRPC وأدوات تزامن واختبار جيدة. Laravel يخدم REST والمستخدمين والمنتجات والطلبات، وGo يتولى مخزونًا مستقلًا. تغيير التطبيق إلى Laravel لا يفرض تغيير Go.

ما الفرق بين REST وgRPC؟ المتصفح وPostman يتعاملان مع REST/JSON للتطبيق؛ Laravel يتعامل مع المستودع عبر gRPC/Protobuf. Postman يستطيع اختبار الطرفين.

لماذا قاعدتان؟ كل خدمة تملك بياناتها. التواصل عبر العقد، ولا يكتب Laravel في جداول مخزون Go. هذا يفصل مسؤوليات الخدمة ويتيح اختبار تعطلها مستقلًا.

لماذا Reserve واحد لكل الطلب؟ حتى تكون السلة all-or-none. إذا نقص أي بند لا تُحجز البنود الأخرى. يجهز AtomicShortage.json مثالًا مباشرًا لذلك.

ماذا لو حُجز المخزون ثم فشل حفظ الطلب؟ يحاول Laravel Release، ويترك سجل استرداد دائمًا للعامل عند التعذر. لا توجد transaction مشتركة بين قاعدتين؛ نستخدم معاملة محلية وتعويضًا قابلًا لإعادة المحاولة.

ما idempotency؟ إعادة العملية نفسها بالمعرّف نفسه لا تضاعف أثرها. Reserve وRelease يستخدمان order_id، وAddStock يستخدم operation_id.

هل 0 OK يعني أن الكمية كافية؟ يعني أن RPC نُفذ بنجاح. نجاح الحجز نفسه يتحدد بـ reserved أو shortages. في REST تصبح قلة المخزون 409.

ماذا يفعل Docker هنا؟ يضم التطبيق وGo وقاعدتي البيانات والعامل في بيئة تشغيل محددة، ويوفر شبكة داخلية وأقراصًا دائمة. اسم الحاوية warehouse هو عنوان Go داخل شبكة التطبيق؛ 127.0.0.1:50051 هو عنوانه من جهاز Postman.

ماذا يثبت اختبار السباق؟ في هذه التجربة، من 20 حسابًا متزامنًا على قطعة واحدة، نجح واحد ورُفض الباقون ولم يصبح المخزون سالبًا. لا يكفي تكرار الطلب 20 مرة بالتتابع لإثبات التعامل مع الطلبات المتزامنة.

هل الإشعارات تعمل؟ تُحفظ عند إنشاء الطلب وتغيير حالته، ويقرأها صاحب الطلب فقط. الواجهة تستعلم كل 10 ثوانٍ أثناء نشاط التبويب؛ ليست WebSocket ولا رسائل هاتف. اختبارات REST تتحقق من إشعار الإنشاء والإلغاء.

8. ما تحقق منه في هذه الحزمة

- مجموعتا Postman تعملان عبر Newman على تطبيق Laravel وخادم Go وقاعدة PostgreSQL: 8 طلبات/13 تحققًا و23 طلبًا/28 تحققًا، دون إخفاقات.
- جُرّبت مجموعة REST أيضًا ببيئة قديمة ذات عنوان وكلمة مرور خاطئين؛ استخدمت بيانات مجموعتها الصحيحة ونجحت. هذا يعالج مشكلة اختيار البيئة السابقة.
- نجح السباق عبر 20 حسابًا مختلفًا: نجاح واحد 201 و19 رفضًا 409 ورصيد صفر.
- نجح اختبار القبول الشامل، بما فيه نقص سلة متعددة المنتجات، وفشل الحفظ والتعويض، وانقطاع المستودع والمهلة ورفض الصلاحيات.
- كانت اختبارات الخادم بعمليات محلية، وجرى التحقق من Compose. لا يتوفر Docker daemon في بيئة إعداد الحزمة؛ أمر Start-Shop يجري البناء وفحص الخدمات الفعليين على جهاز العرض. إعداد Postman Desktop وملف الصورة يجهزان على جهازك قبل المقابلة.

مراجع الأدوات

- Postman Collection Runner: https://learning.postman.com/docs/tests-and-scripts/running-collections/intro-to-collection-runs/
- Postman gRPC: https://learning.postman.com/docs/use/send-requests/protocols/grpc/grpc-request-interface
- Docker Compose up: https://docs.docker.com/reference/cli/docker/compose/up/
- Docker Compose stop: https://docs.docker.com/reference/cli/docker/compose/stop/
- Docker Compose down: https://docs.docker.com/reference/cli/docker/compose/down/